PACKAGING

A.SCHUTZ

Qu’est qu’un package python

Un package est une manière d'organiser des modules Python (fichiers .py) dans une hiérarchie de dossiers pour les rendre identifiables, importables et reproductibles.

C'est l'unité de base de la distribution en Python.

```
mon_projet/
├── pyproject.toml
└── mon_projet/
    ├── __init__.py  # La porte d'entrée obligatoire
    ├── model.py     # Module interne
    └── train.py     # Module interne
```

L'Installation : Un package ne se "copie-colle" pas. Il s'installe (via uv ou pip) pour devenir disponible partout.

La Version : Un package possède un numéro de version unique.

La bibliothèque (package) à le même nom que le dépôt Git

__init__.py

C'est ici que l'on décide de ce qui est Public et de ce qui est Privé

Sans `__init__.py` : `from mon_package.sous_dossier.calculs import ma_fonction`

Avec `__init__.py` : `from mon_package import ma_fonction`

Si on inclus dans `__init__.py` : `from .sous_dossier.calculs import ma_fonction`

On peut aussi indiquer la version du code en mettant: `__version__ = "1.0.0"`

`__all__`

C'est un paramètre optionel du fichier `__init__.py` il permet de définir ce qui sort vraiment.

Par exemple on peut dans le `__init__.py` avoir besoin d'une fonction (f1) pour une autre fonction (f2) sans vouloir que f1 soit accessible dans le module.

Dans ce cas dans le `__init__.py` on aura `__all__ = ["f2", "autre_fcts"]` mais pas f1

C'est aussi ce qui sort d'un : `from mon_package import *`

Structure d’un Package

```
mon_projet/
├── pyproject.toml
└── src/
    ├── __init__.py  # La porte d'entrée obligatoire
    ├── model.py     # Module interne
    └── train.py     # Module interne
```

Votre package s’appelle `mon_projet`

à la racine il n’y a aucun code:

- docker
- cicd
- readme etc..)

Le `Pyproject.toml` qui va définir l’identité du package

toml

```toml
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "mon_projet_mlops"
dynamic = ["version"] # Version dynamique
description = "Pipeline d'entraînement"
readme = "README.md"
requires-python = ">=3.10"
license = "MIT"
authors = [
    { name = "Nom", email = "mon@email.com" },
    { name = "Nom", email = "mon@email.com" },
]

# --- DÉPENDANCES DE PROD (le strict minimum)
dependencies = [
    "pandas",
    "scikit-learn",
    "prefect>=3.0.0",
    "mlflow",
    "pydantic",
]
```

```toml
# --- DÉPENDANCES OPTIONNELLES (Extras)
# Installable via: pip install ".[dev]" ou ".[test]"
[project.optional-dependencies]
test = [
    "pytest",
    "pytest-cov",
]
dev = [
    "ruff",          # Linter & Formatter
    "ipykernel",     # Pour les notebooks
    "pre-commit",    # Pour automatiser les checks avant git commit
]
doc = [
    "furo",          # Thème pour Sphinx
    "sphinx",
]

# --- SCRIPTS (Entry points)
[project.scripts]
train-model = "mon_projet_mlops.main:run_train"
api-start = "mon_projet_mlops.api:start"

# --- CONFIGURATION HATCH (Version dynamique)
[tool.hatch.version]
path = "src/mon_projet_mlops/__init__.py"

# --- CONFIGURATION DES OUTILS (Optionnel mais recommandé)
[tool.ruff]
line-length = 88
select = ["E", "F", "I"] # Erreurs, Flakes, Isort (tri des imports)

[tool.pytest.ini_options]
testpaths = ["tests"]
```

uv pip install -e .

Quand on a un module et qu'on fait un `pip install module` les modifications faites dans le module ne sont appliquées que si on refait un `pip install`.

L'option du `pip install -e .` fait que les modifications sont prises en compte au fur et à mesure.

Cela permet de développer et tester le module en même temps.

uv build

```
- name: Build Package
        run: uv build
```

Le "Build" transforme le dossier `src/` en deux fichiers standardisés que Python sait lire partout.

- Le SDist (Source Distribution) : Un fichier `.tar.gz` qui contient ton code source.
- Le Wheel (`.whl`) : Un format binaire (compressé) prêt à l'emploi. C'est ce que pip télécharge.

Un dossier `dist/` apparaît à la racine avec tes fichiers packagés.

Installer le build : `uv pip install mon_projet/dist/mon_package-0.1.0-py3-none-any.whl`

Tester l'import : `python -c "import mon_package;print(mon_package.__version__)"`

Publier

Github (lent): `pip install git+https://github.com/ton-nom/mon-projet.git`

Pypi : Packaging Python Projects

Github Release : CICD ou Manuel

```yaml
name: Release Automatique

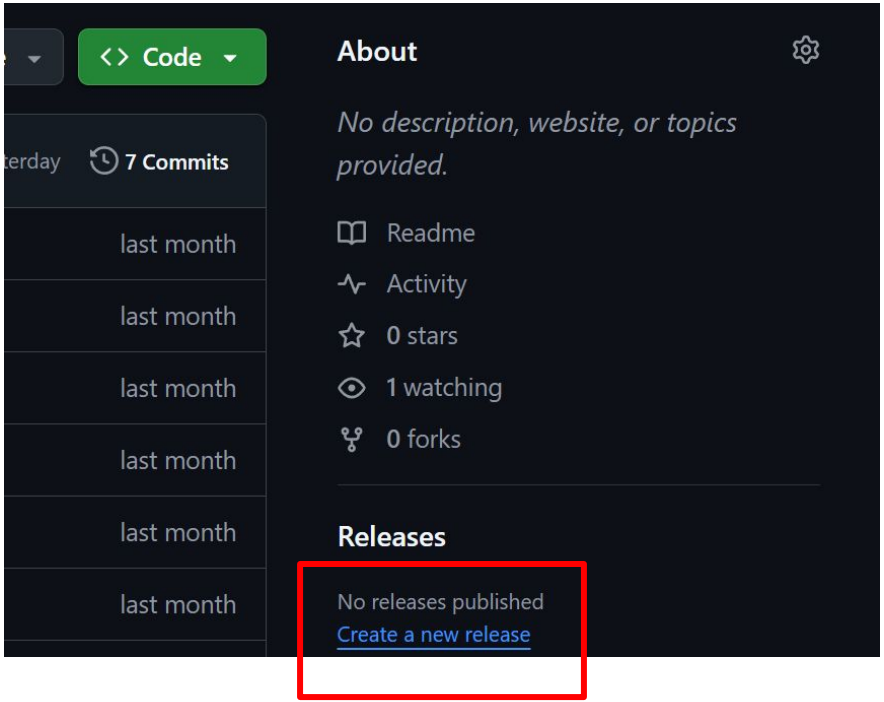
on:
  push:
    tags:
      - 'v*' # Se déclenche pour un tag

jobs:
  build-and-release:
    runs-on: ubuntu-latest
    permissions:
      contents: write # Obligatoire
    steps:
      - uses: actions/checkout@v4

      - name: Install uv
        uses: astral-sh/setup-uv@v3

      - name: Build
        run: uv build

      - name: Create GitHub Release
        uses: softprops/action-gh-release@v2
        with:
          files: dist/*.whl
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```



- onglet Settings de ton dépôt.
- le menu de gauche: sur Actions > General.
- En bas, section Workflow permissions.
- Cocher "Read and write permissions".
- Save.

Annexe Streamlit

```
app_front/
├── .streamlit/
│   └── config.toml   # Le style (Couleurs, Polices, Arrondis)
├── main.py           # Le code (Logique, API calls)
├── pyproject.toml
└── Dockerfile
```

```toml
primaryColor = "#7C4DFF"
backgroundColor = "#0E1117"
secondaryBackgroundColor = "#262730"
textColor = "#FAFAFA"
font = "sans serif"
# Pour un look moderne arrondi
borderRadius = "0.5rem"
```